BOOKISH

A Role-Based Online Book Marketplace

Web-Based Development - Project Report

Technology Stack: React 18 • Node.js / Express • MongoDB • Stripe • Redis • Docker

Architecture: RESTful API • Domain-Driven Design • Microservice-Ready Monolith

Date: April 2026

Group 53

Ayush Pratap Singh - S20230010033

Piyush Kumar - S20230010186

Daivik Wadhwani - S20230010064

Ujjwal Singh - S20230010245

Gugulothu Nithin - S20230010099

Under the guidance of:

Dr. Himangshu Sarma

Dr. B N Pavan Kumar

Table of Contents

1. **Project Description**

2. **Domain-Driven Design**

    a. Bounded Contexts

    b. Context Mappings

    c. Entities, Value Objects & Services

    d. Cardinality Ratios

    e. Aggregates

3. **Technology Stack Summary**

1. Project Description

Bookish is a full-stack online book marketplace that connects sellers with buyers through a secure and feature-rich web platform. The frontend is built with React 18, the backend runs on Node.js with Express, MongoDB handles data persistence, and Stripe takes care of payment processing.

The platform supports five user roles: **Buyer**, **Seller**, **Admin**, **Moderator**, and **Employee**, each with its own dashboard and set of workflows. Buyers can browse books, manage a cart, place orders, subscribe to premium plans for digital library access (complete with an in-app PDF reader, highlights, annotations, and bookmarks), watch and upload book review videos, and file complaints. Sellers list books (both physical and digital), manage inventory, track orders, monitor revenue, and respond to customer issues. Admins have full oversight of the platform, and Moderators along with Employees handle content moderation, including seller verification, book approval, complaint resolution, and order escalations.

Authentication is session-based using Passport.js with bcrypt for password hashing, and authorization is enforced through role-based middleware. Media is stored on Cloudinary, file uploads go through Multer, and subscriptions are tiered into Free, Premium, and Premium Plus plans. On the frontend, Redux with Thunk manages global state, React Router handles navigation, Tailwind CSS provides responsive styling, and Framer Motion powers animations. The overall architecture follows RESTful conventions with Swagger/OpenAPI documentation, uses database indexing and Redis caching to boost performance, integrates Apache Solr for better search, runs inside Docker containers, and relies on a GitHub Actions CI/CD pipeline for automated testing and deployment to Vercel.

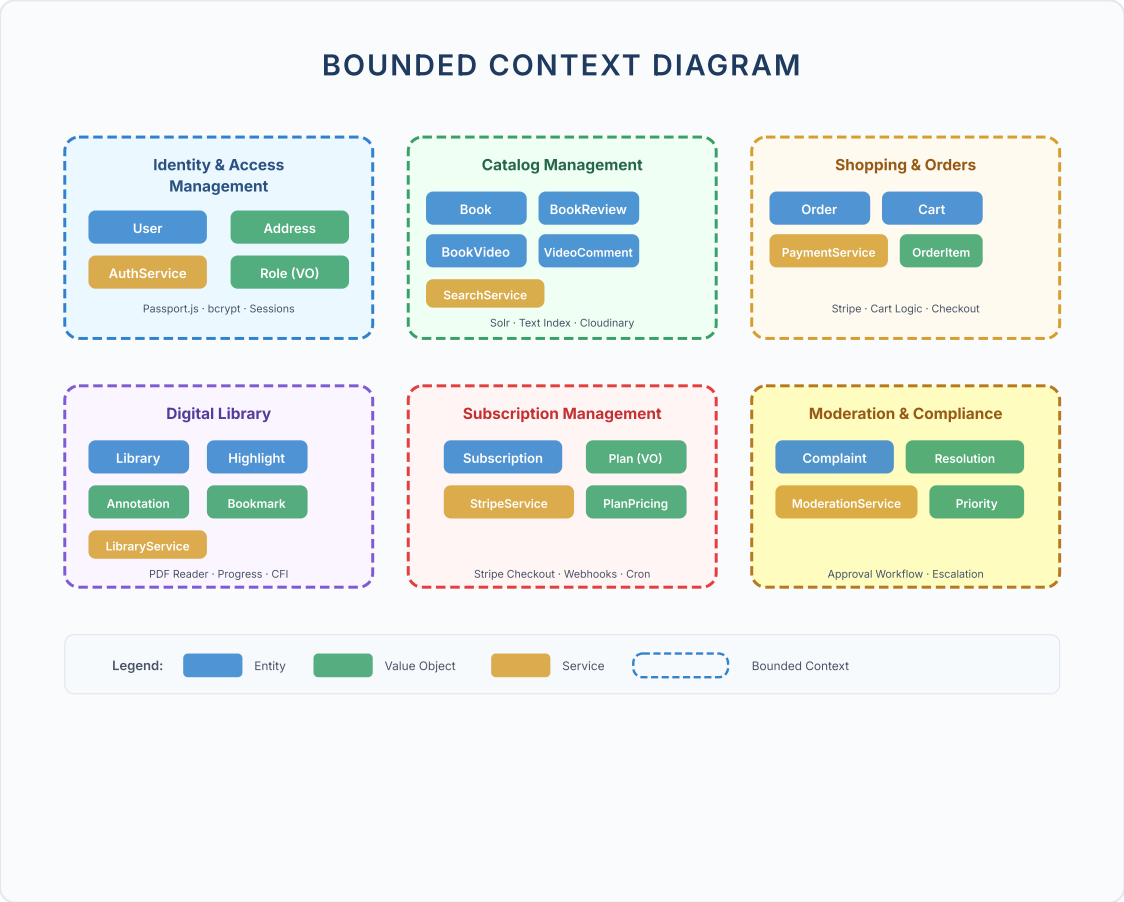
11 DOMAIN MODELS	**80+** API ENDPOINTS	**5** USER ROLES
40+ REACT PAGES	**6** BOUNDED CONTEXTS	**7** AGGREGATES

2. Domain-Driven Design

We structured the Bookish platform using Domain-Driven Design (DDD) principles. The entire domain is broken down into six bounded contexts, where each one covers a specific area of business logic with its own boundaries, entities, value objects, and domain services.

2a. Bounded Contexts

The diagram below shows the six bounded contexts we identified in the Bookish domain. Each context owns its own entities and is responsible for enforcing its own rules.



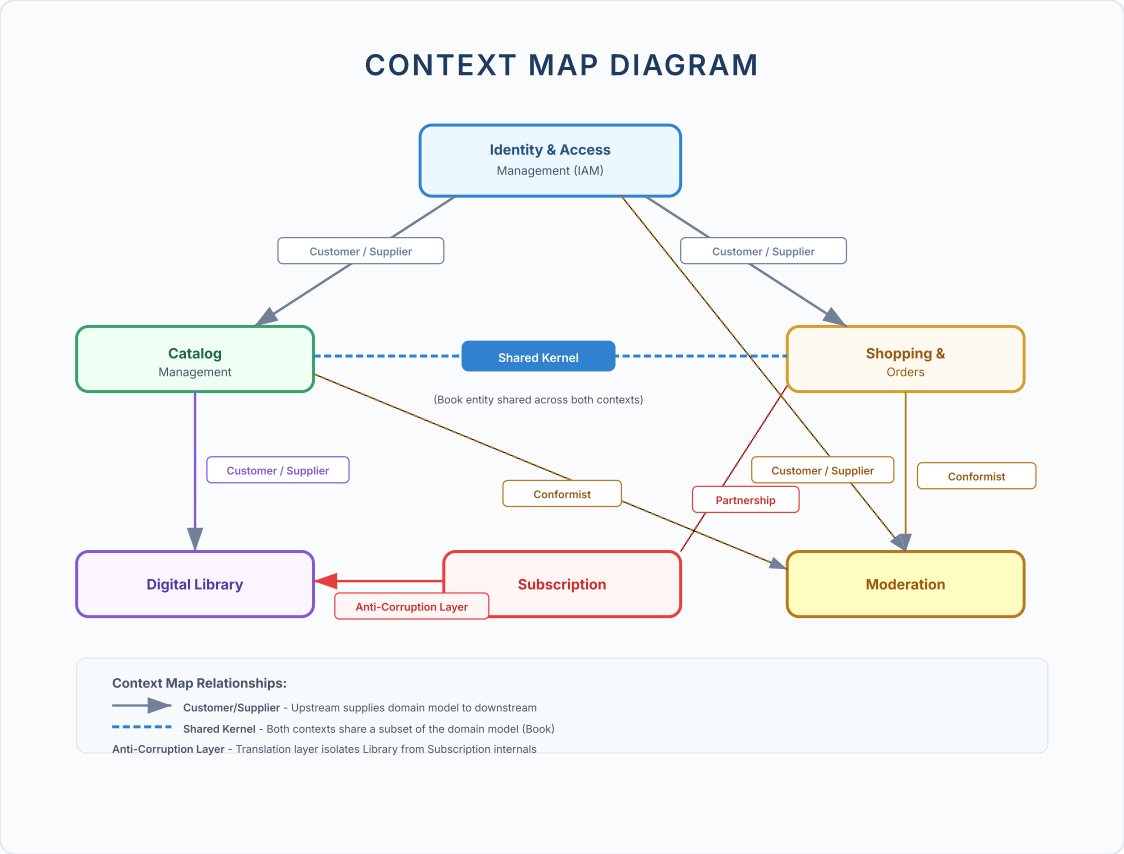
Bounded Context Descriptions

BOUNDED CONTEXT	RESPONSIBILITY	KEY DOMAIN CONCEPTS
Identity & Access	Handles user registration, login/logout (Passport.js with	User, Address, Role, VerificationStatus,

Management	sessions), role-based authorization through middleware, profile updates, and address management	Password hashing
Catalog Management	Covers book listings, searching and browsing, book reviews with ratings, video content for books, and video comments	Book, BookReview, BookVideo, VideoComment, ISBN, Genre, Format
Shopping & Orders	Manages the shopping cart, checkout flow, the full order lifecycle, Stripe payment processing, order tracking, returns, and cancellations	Order, Cart, OrderItem, ShippingAddress, PaymentDetails, OrderStatus
Digital Library	Provides subscription-gated access to digital reading, including a PDF viewer, reading progress tracking, highlights, annotations, and bookmarks	Library, Highlight, Annotation, Bookmark, ReadingProgress, CFI position
Subscription Management	Manages subscription plans (Free, Premium, Premium Plus), Stripe checkout sessions, webhook handling, the subscription lifecycle, and auto-renewal through cron jobs	Subscription, Plan, PlanPricing, StripeSubscriptionId
Moderation & Compliance	Takes care of content moderation (book approvals), user verification for sellers/employees/moderators, complaint management, and escalation workflows	Complaint, Resolution, ComplaintCategory, Priority, ApprovalStatus

2b. Context Mappings

Context maps lay out how the different bounded contexts relate to each other and how they exchange data. The diagram below shows the communication and dependency patterns across all of Bookish's contexts.



Context Mapping Relationships

UPSTREAM	DOWNSTREAM	PATTERN	DESCRIPTION
Identity & Access	Catalog	Customer / Supplier	The User entity (as a Seller) is referenced by Book.seller. IAM owns the user identity and provides it to the Catalog context.
Identity & Access	Shopping & Orders	Customer / Supplier	The User (as a Buyer) places orders. IAM provides the authenticated user to the Order context.

Catalog and Shopping	Bidirectional	Shared Kernel	The Book entity is shared between Catalog (for listings) and Shopping (for order items and cart). Both sides read from the same Book data.
Catalog	Digital Library	Customer / Supplier	The Library references Book entities for reading content like PDFs and metadata. Catalog supplies this data.
Subscription	Digital Library	Anti-Corruption Layer	The Library checks subscription status through middleware. An ACL translates the Stripe subscription state into a simple access gate.
Identity & Access	Moderation	Customer / Supplier	Moderation verifies users such as sellers and employees. IAM provides the User entities needed for verification workflows.
Catalog	Moderation	Conformist	Moderation follows the Catalog's Book model for its approval and rejection workflows.
Shopping	Moderation	Conformist	Moderation views and manages Orders by conforming to the existing Order model structure.
Shopping and Subscription	Bidirectional	Partnership	Both share the Stripe payment infrastructure. Subscription checkout and order payments go through the same Stripe integration.

2c. Entities, Value Objects & Services

Every bounded context has its own set of entities (objects that carry a unique identity), value objects (immutable descriptors with no identity of their own), and domain services (stateless operations that don't naturally fit inside any single entity).

1. Identity & Access Management Context

TYPE	NAME	KEY ATTRIBUTES / RESPONSIBILITY
Entity	**User**	name, email, password (hashed), role, verificationStatus, avatar, phone, isVerified, managedBy, recentlyViewed, createdAt
Entity	**Address**	user (ref), name, phone, street, city, state, zipCode, country, isDefault
Value Object	Role	Enum: buyer, seller, admin, moderator, employee
Value Object	VerificationStatus	Enum: pending, approved, rejected
Value Object	Email	Unique, validated string format
Value Object	Password	Hashed with bcrypt (12 rounds), select: false
Value Object	EmbeddedAddress	street, city, state, zipCode, country (on User subdoc)
Value Object	ResetToken	resetPasswordToken, resetPasswordExpire
Service	AuthService	Register, login (Passport Local), logout, session management, password hashing/comparison
Service	UserService	Profile CRUD, avatar upload (Cloudinary), role management, user search

Service	AddressService	CRUD addresses, set default address

2. Catalog Management Context

TYPE	NAME	KEY ATTRIBUTES / RESPONSIBILITY
Entity	**Book**	title, author, isbn (unique), description, price, discountPrice, coverImage, publisher, publishedDate, pageCount, language, genres[], condition, format, epubFile, seller (ref), stock, approvalStatus, rating, reviewCount
Entity	**BookReview**	book (ref), user (ref), rating (1-5), reviewText, images[], video, helpfulCount, isVerifiedPurchase, isApproved, replies[]
Entity	**BookVideo**	book (ref), user (ref), title, description, videoUrl, thumbnailUrl, views, likes[], duration, tags[]
Entity	**VideoComment**	video (ref), user (ref), content, likes[]
Value Object	ISBN	Unique book identifier string
Value Object	Genre	Array of genre strings (min 1)
Value Object	BookCondition	Enum: new, used
Value Object	BookFormat	Enum: paperback, hardcover, ebook, audiobook
Value Object	ApprovalStatus	Enum: pending, approved, rejected + rejectionReason, dates
Value Object	ReviewMedia	images[{url, publicId, caption}], video{videoUrl, thumbnailUrl, duration}
Value Object	ReviewReply	user (ref), text, createdAt

Service	BookService	CRUD books, search/filter/browse, ISBN lookup, stock management
Service	ReviewService	Create/delete reviews, helpful votes, verified purchase check
Service	VideoService	Upload/delete videos, likes, comments, video feed
Service	SearchService	Full-text search (MongoDB text index + Solr), faceted filtering

3. Shopping & Orders Context

TYPE	NAME	KEY ATTRIBUTES / RESPONSIBILITY
Entity	**Order**	buyer (ref), orderId (unique), items[], totalAmount, subtotal, tax, shippingCost, adminCommission, sellerRevenue, paymentMethod, paymentStatus, shippingAddress, orderStatus, trackingNumber, statusHistory[], returnRequest
Entity	**Cart**	user (ref, unique), items[], savedItems[], totalAmount
Value Object	OrderItem	book (ref), seller (ref), title, author, coverImage, quantity, price
Value Object	CartItem	book (ref), quantity, price
Value Object	ShippingAddress	name, address, city, state, pincode, phone
Value Object	PaymentDetails	transactionId, paymentDate, amount
Value Object	OrderStatus	Enum: pending, confirmed, shipped, delivered, cancelled, returned
Value Object	TrackingInfo	trackingNumber, carrier, estimated delivery

Value Object	StatusHistoryEntry	status, timestamp, note
Service	OrderService	Create order, update status, cancel, return request, commission calculation
Service	CartService	Add/remove/update items, save for later, move to cart, clear cart
Service	PaymentService	Stripe PaymentIntent creation, payment verification, refund processing

4. Digital Library Context

TYPE	NAME	KEY ATTRIBUTES / RESPONSIBILITY
Entity	**Library**	user (ref), items[] containing book refs with progress, annotations, bookmarks
Entity	**Highlight**	userId (ref), bookId, text, page, color, position (startOffset, endOffset, boundingRect)
Value Object	ReadingProgress	progress (0-100), currentPage, cfi (content fragment identifier)
Value Object	Annotation	cfi, text, note, color, createdAt, updatedAt
Value Object	Bookmark	isBookmarked, bookmarkPage
Value Object	HighlightPosition	startOffset, endOffset, boundingRect
Value Object	LibraryItemMeta	accessCount, lastAccessed, addedAt
Service	LibraryService	Add/remove books, update progress, manage annotations/bookmarks
Service	HighlightService	CRUD highlights per book per user
Service	PDFStreamService	Proxy and stream PDF content from Cloudinary URLs

5. Subscription Management Context

TYPE	NAME	KEY ATTRIBUTES / RESPONSIBILITY
Entity	**Subscription**	user (ref), plan, startDate, endDate, renewalDate, isActive, autoRenew, stripeSubscriptionId, stripeCustomerId
Value Object	Plan	Enum: free, premium, premium_plus
Value Object	PlanPricing	Price tiers, features, limits per plan
Value Object	SubscriptionPaymentDetails	Stripe session info, payment method, amount
Value Object	SubscriptionPeriod	startDate, endDate, renewalDate
Service	SubscriptionService	Plan status check, create/cancel subscription, verify session
Service	StripeWebhookService	Handle Stripe webhook events, update subscription state
Service	RenewalCronService	Automated renewal checks and expiration via node-cron

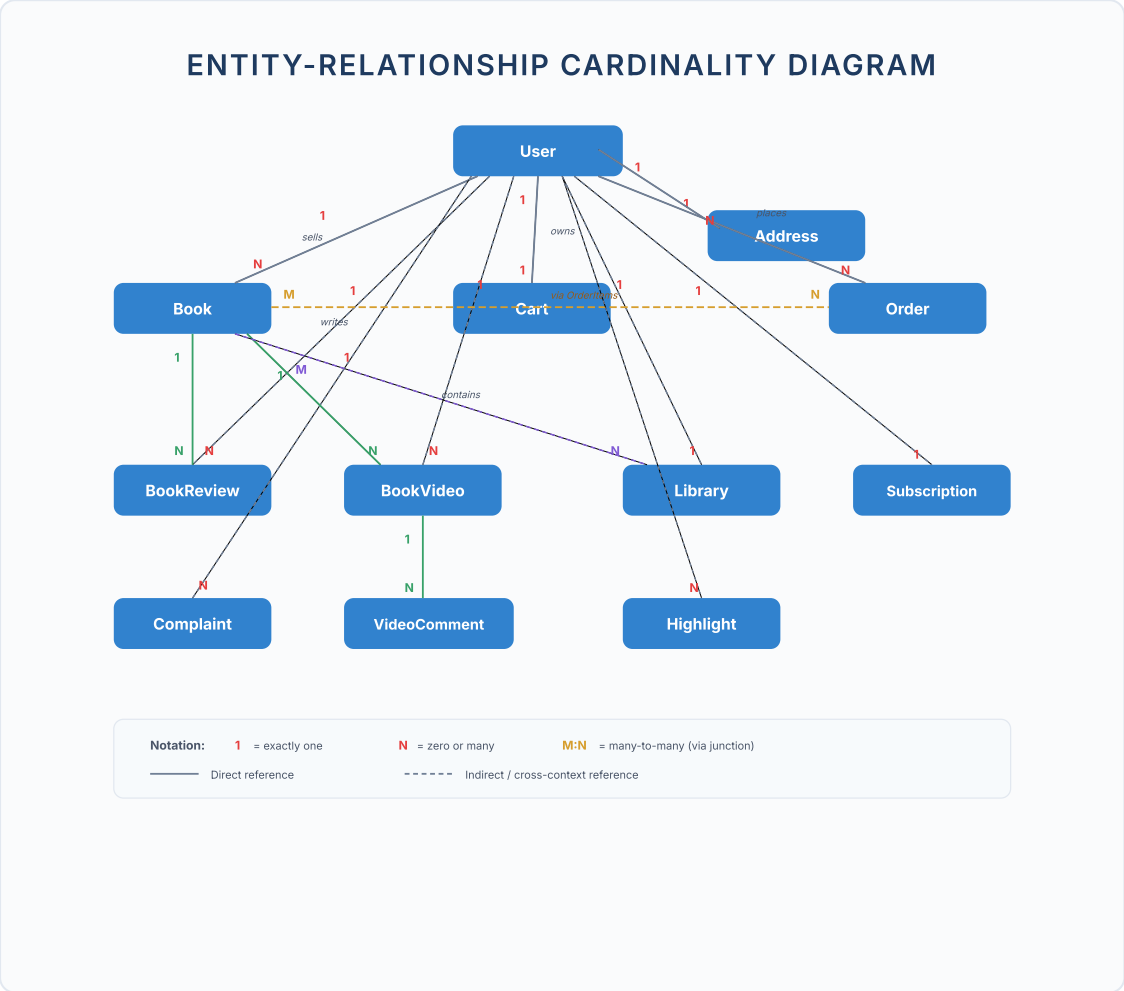
6. Moderation & Compliance Context

TYPE	NAME	KEY ATTRIBUTES / RESPONSIBILITY
Entity	**Complaint**	subject, description, category, user (ref), order (ref), book (ref), status, priority, assignedTo (ref), comments[], resolution, adminResponse
Value Object	ComplaintCategory	Long enum of categories (order issue, payment, content, etc.)

Value Object	Priority	Enum: low, medium, high, critical
Value Object	Resolution	action, details, resolvedAt, resolvedBy (ref)
Value Object	ComplaintComment	user (ref), text, role, createdAt
Value Object	GuestInfo	name, email, phone (for unauthenticated complaints via contact form)
Service	ModerationService	Book approval/rejection, user verification, lock/unlock workflow
Service	ComplaintService	File complaint, assign, comment thread, escalation, resolution
Service	VerificationService	Approve/reject sellers, employees, moderators; manage verification status

2d. Cardinality Ratios

The table below lists the cardinality relationships between all entities in our domain model. These are taken directly from how the Mongoose schemas are defined in the codebase.



Cardinality Ratio Table

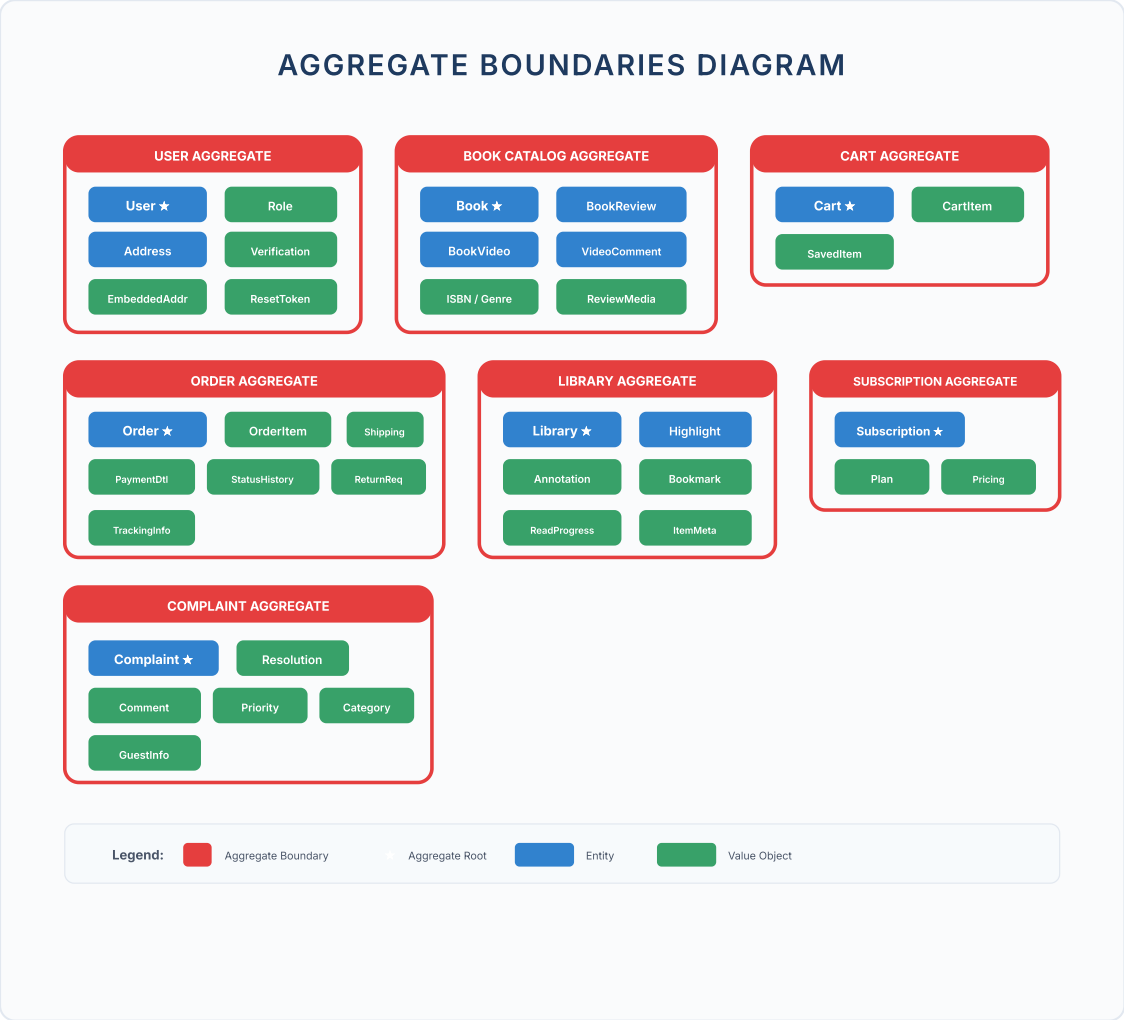
RELATIONSHIP	CARDINALITY	CONSTRAINT / NOTES
User to Book (as seller)	**1 : N**	A seller can list many books, but each book belongs to exactly one seller
User to Cart	**1 : 1**	There is a unique constraint on Cart.user, so each user gets one cart

User to Order (as buyer)	**1 : N**	A buyer can place as many orders as they want
User to Address	**1 : N**	Users can save multiple addresses
User to BookReview	**1 : N**	A user can write many reviews, but only one per book (compound unique index)
User to BookVideo	**1 : N**	A user can upload multiple book videos
User to Library	**1 : 1**	Each user has one library document that holds multiple items
User to Subscription	**1 : 1**	A user can only have one active subscription at a time
User to Complaint	**1 : N**	A user can file multiple complaints
User to Highlight	**1 : N**	A user can create many highlights across different books
User to User (managedBy)	**N : 1**	Self-referential relationship where employees are managed by a moderator or admin
Book to BookReview	**1 : N**	Each book can have many reviews
Book to BookVideo	**1 : N**	Each book can have many associated videos
Book to Order	**M : N**	Linked through OrderItem subdocuments. A book can appear in many orders, and an order can contain many books.
Book to Library	**M : N**	Linked through Library.items subdocuments. A book can exist in many user libraries.
Book to Cart	**M : N**	Linked through Cart.items subdocuments

BookVideo to VideoComment	**1 : N**	Each video can have many comments
BookReview to User (unique compound)	**N : 1**	Enforced by a unique index on (user, book), so only one review per user per book
Complaint to Order	**N : 1**	Optional reference. A complaint may be tied to one order.
Complaint to Book	**N : 1**	Optional reference. A complaint may be tied to one book.

2e. Aggregates

Aggregates set the consistency boundaries within the domain. Each aggregate has a root entity that acts as the single point of access for all the entities and value objects inside it, which keeps transactions consistent.



4. Technology Stack Summary

12.1 Complete Technology Map

LAYER	TECHNOLOGY	VERSION	PURPOSE
Backend	Node.js	20.x	Runtime environment
	Express.js	4.x	Web framework
	Mongoose	8.x	MongoDB ODM
	Passport.js	0.7.x	Authentication (Local strategy)
	Stripe SDK	14.x	Payment processing
	Cloudinary	2.x	Media upload & CDN
	Multer	1.x	File upload handling
	Helmet	7.x	Security headers
	swagger-jsdoc + swagger-ui-express	6.x / 5.x	API documentation
	node-cron	3.x	Scheduled tasks (subscription renewal)
Frontend	React	18.x	UI library
	Vite	6.x	Build tool & dev server

	React Router	6.x	Client-side routing
	Redux + Thunk + Persist	5.x	State management
	Tailwind CSS	3.x	Utility-first CSS
	Axios	1.x	HTTP client
	React Hook Form + Zod	7.x / 3.x	Form handling & validation
	Stripe Elements	2.x	Payment UI
	Framer Motion	11.x	Animations
Database & Cache	MongoDB	7.x	Primary data store
	MongoDB Atlas	-	Managed cloud database
	Redis	7.x	Caching layer
	Apache Solr	9.x	Full-text search engine
DevOps	Docker + Compose	24.x / 2.x	Containerization
	GitHub Actions	-	CI/CD pipeline
	Vercel	-	Production deployment and hosting
Testing	Jest + Supertest	29.x / 6.x	Backend testing
	Vitest + React Testing Library	1.x / 14.x	Frontend testing
	Istanbul / c8	-	Code coverage
Monitoring	Morgan + rotating-file-stream	1.x	Request logging

	express-rate-limit	7.x	Rate limiting

Bookish - A Role-Based Online Book Marketplace

Web-Based Development Project Report • Group 53 • April 2026

All diagrams represent the domain model as implemented in the codebase.